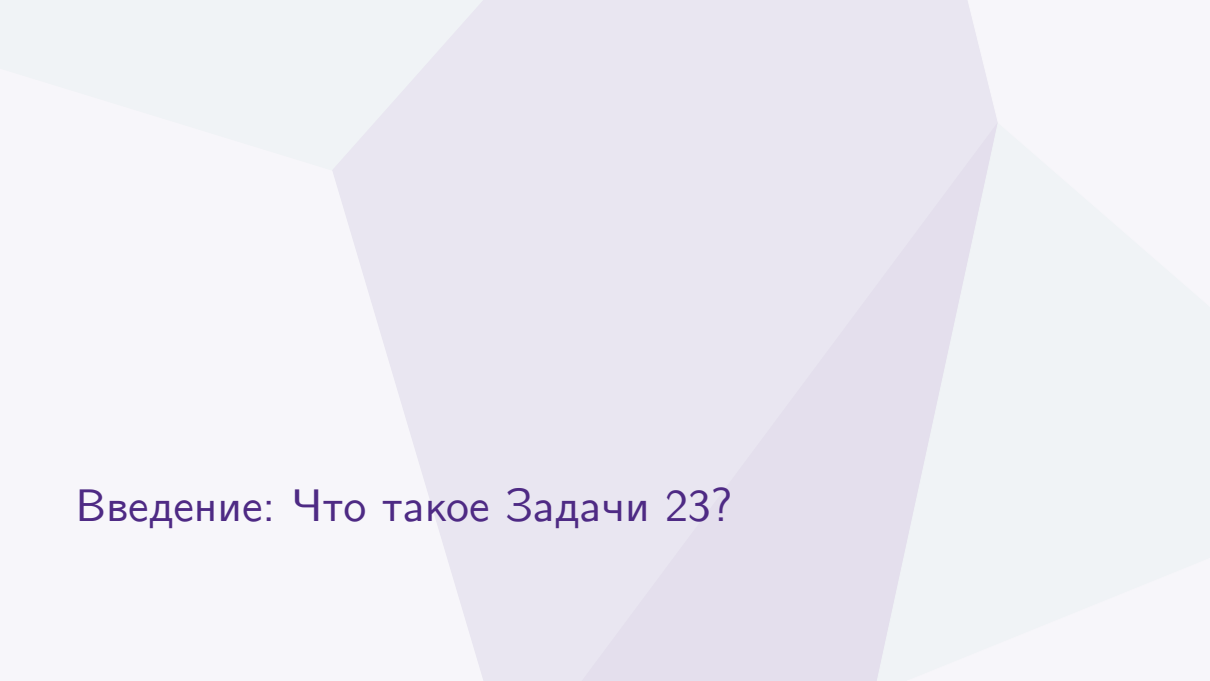


Финальная подготовка к ЕГЭ. Информатика. № 23

Информатика ЕГЭ



Введение: Что такое Задачи 23?

О чем Задание 23?

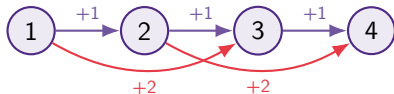
Задание 23 проверяет ваше умение считать количество различных вариантов (программ), с помощью которых исполнитель может превратить одно число в другое.

Например: Исполнитель переходит от числа **1** к числу **4**. Команды $\rightarrow +1$ и $+2$.

Как мы можем туда добраться?

- $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 (+1, +1, +1)$
- $1 \rightarrow 2 \rightarrow 4 (+1, +2)$
- $1 \rightarrow 3 \rightarrow 4 (+2, +1)$

Всего 3 программы!



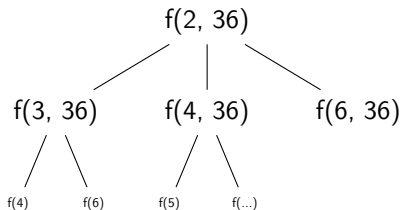
Почему мы не можем перебирать всё руками?

В реальном экзамене мы идем не от 1 к 4, а, например, **от 2 к 36**.

Там могут быть тысячи вариантов!

Поэтому мы используем **Рекурсию**. Мы пишем функцию $f(\text{start}, \text{end})$, которая:

- Вызывает саму себя для каждого возможного следующего шага.
- Складывает количество найденных путей.



Разбор рекуррентного уравнения

Структура любой рекурсивной функции

Любая функция для Задания 23 делится на три блока:

1. **Случай «Промеха»:** Если мы прибавляли так много, что перелетели цель ($\text{start} > \text{end}$). Мы возвращаем **0** (тупик, путей нет).
2. **Случай «Попадания»:** Если мы оказались ровно в конечной точке ($\text{start} == \text{end}$). Мы возвращаем **1** (мы нашли одну успешную программу!).
3. **Случай «Продолжения»:** Мы прибавляем/умножаем текущее число и запускаем функцию от всех полученных вариантов, **складывая их результаты**.

```
def f(start, end):  
    if start > end: return 0      # 1. Промех  
    if start == end: return 1    # 2. Попадание  
    # 3. Продолжение: команды (+1) и (*2)  
    return f(start + 1, end) + f(start * 2, end)
```

Особые случаи: Обязательный узел

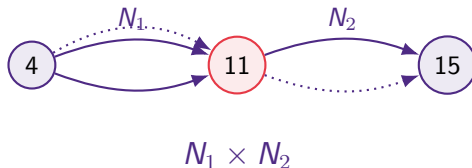
Обязательный узел

Например: Программа преобразует 4 в 15, при этом траектория **содержит число 11**.

Это значит, что наша дорога разбивается на два независимых отрезка:

1. От 4 до 11.
2. От 11 до 15.

Мы должны взять количество путей для первого отрезка и **умножить** на количество путей для второго.



Особые случаи: Избегаемый узел

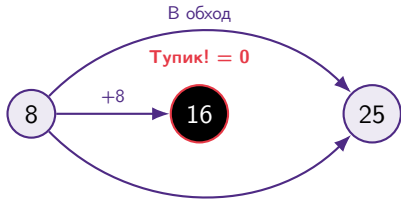
Избегаемый узел

Например: Программа преобразует 8 в 25,
не содержит число 16.

Это значит, что узел «16» превращается в черную дыру. Как только траектория заходит в 16, она аннулируется.

В коде мы просто подмешиваем этот запрет в условие «Промеха»:

```
if start > end or start == 16:  
    return 0
```



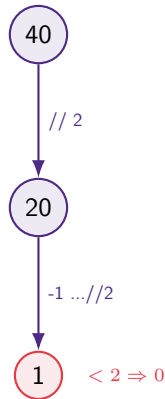
Особые случаи: Отрицательные действия (-1, //2)

Отрицательные действия

Иногда мы идём от большего к меньшему, например **от 40 к 2**. Действия: -1 и //2 (целочисленное деление на 2).

В коде мы должны **поменять знаки отсечения**. Так как мы идем вниз, мы промажем, если `start < end`.

```
def f(start, end):  
    if start < end: # Мы упали слишком низко  
        return 0  
    if start == end:  
        return 1  
  
    return f(start - 1, end) + f(start // 2, end)
```





Решение задач

Задача 1: Разминка рук (Пишем ручкой!)

Задача

Исполнитель преобразует число на экране. У исполнителя есть три команды, которым присвоены номера: 1. Прибавить 1; 2. Прибавить 2; 3. Умножить на 2. Сколько существует программ, для которых при исходном числе 2 результатом является число 20? **Напишите полный код функции Python вручную.**

Решение Задачи 1 (Код)

```
def f(start, end):  
    # Если перелетели цель - возвращаем 0 путей  
    if start > end:  
        return 0  
    # Базовый случай: мы попали ровно в цель  
    if start == end:  
        return 1  
  
    # Рекурсивный переход: перебираем варианты действий  
    return f(start + 1, end) + f(start + 2, end) + f(start * 2, end)  
  
# Вызываем функцию от старта (2) до финиша (20)  
print(f(2, 20))
```

Решение Задачи 1 (Ответ)

Ответ

Ответ: 8118

Задача 2

Задача

Исполнитель преобразует число на экране. У исполнителя есть три команды: 1. Прибавить 1; 2. Умножить на 2; 3. Умножить на 3. Сколько существует программ, для которых при исходном числе 2 результатом является число 36?

Заполните пропуски в коде решения на рабочем листе.

Решение Задачи 2 (Код)

```
def f(start, end):  
    if start > end:  
        return 0  
    if start == end:  
        return 1  
  
    # Команды: Прибавить 1, умножить на 2, умножить на 3  
    return f(start + 1, end) + f(start * 2, end) + f(start * 3, end)  
  
print(f(2, 36))
```

Ответ

Ответ: 268

Задача 3 (ФИПИ)

ФИПИ

У исполнителя есть три команды: А. Прибавить 1; В. Прибавить 2; С. Умножить на 2.

Сколько существует программ, которые преобразуют число 4 в 15, при этом траектория программы содержит числа 11 и 13?



Решение Задачи 3 (Код)

```
def f(start, end):  
    if start > end:  
        return 0  
    if start == end:  
        return 1  
    return f(start + 1, end) + f(start + 2, end) + f(start * 2, end)  
  
# Траектория должна ОБЯЗАТЕЛЬНО пройти через 11 и 13.  
# Перемножаем количество путей на каждом из отрезков пути  
res_1 = f(4, 11)    # от старта до первого узла  
res_2 = f(11, 13)   # между обязательными узлами  
res_3 = f(13, 15)   # от последнего узла до финиша  
  
print(res_1 * res_2 * res_3)
```

Решение Задачи 3 (Ответ)

Ответ

Ответ: 100

Задача 4 (ФИПИ)

ФИПИ

У исполнителя есть команды: 1. Прибавить 1; 2. Прибавить 2; 3. Умножить на 3. Сколько существует программ, которые преобразуют 8 в 25, при этом траектория программы не содержит числа 9 и 16?



Решение Задачи 4 (Код)

```
def f(start, end):  
    # Если превысили цель ИЛИ попали в запрещенный узел - отказ!  
    if start > end or start == 9 or start == 16:  
        return 0  
    if start == end:  
        return 1  
  
    return f(start + 1, end) + f(start + 2, end) + f(start * 3, end)  
  
# Считаем пути напрямую, функция сама обогнет узлы 9 и 16  
print(f(8, 25))
```

Ответ

Ответ: 273

Задача 5

Задача

У исполнителя есть команды: 1. Прибавить 1; 2. Прибавить 2; 3. Умножить на 2. Сколько существует программ, для которых при исходном числе 2 результатом является число 40 и при этом траектория вычислений содержит число 30 и не содержит число 12?

Решение Задачи 5 (Код)

```
def f(start, end):  
    # Отсекаем, если превысили цель ИЛИ попали в избегаемый узел (12)  
    if start > end or start == 12:  
        return 0  
    if start == end:  
        return 1  
    return f(start + 1, end) + f(start + 2, end) + f(start * 2, end)  
  
# Обязательная точка - 30, разбиваем на два отрезка  
# Избегаемая точка - 12 уже( учтена внутри f)  
print(f(2, 30) * f(30, 40))
```

Решение Задачи 5 (Ответ)

Ответ

Ответ: 26853258

Задача 6

Задача

Исполнитель преобразует число на экране. У исполнителя есть три команды: 1. Вычесть 1; 2. Разделить нацело на 2; 3. Разделить нацело на 3. Сколько существует программ, для которых при исходном числе 40 результатом является число 2, и при этом траектория не содержит 10?



Решение Задачи 6 (Код)

```
def f(start, end):  
    # Тк.. мы идем ВНИЗ от( 40 к 2), мы перелетим цель, если станем МЕНЬШЕ  
    if start < end or start == 10:  
        return 0  
    if start == end:  
        return 1  
  
    # Команды: вычесть 1, целочисленное деление на 2, целочисленное деление на 3  
    return f(start - 1, end) + f(start // 2, end) + f(start // 3, end)  
  
print(f(40, 2))
```

Ответ

Ответ: 1159